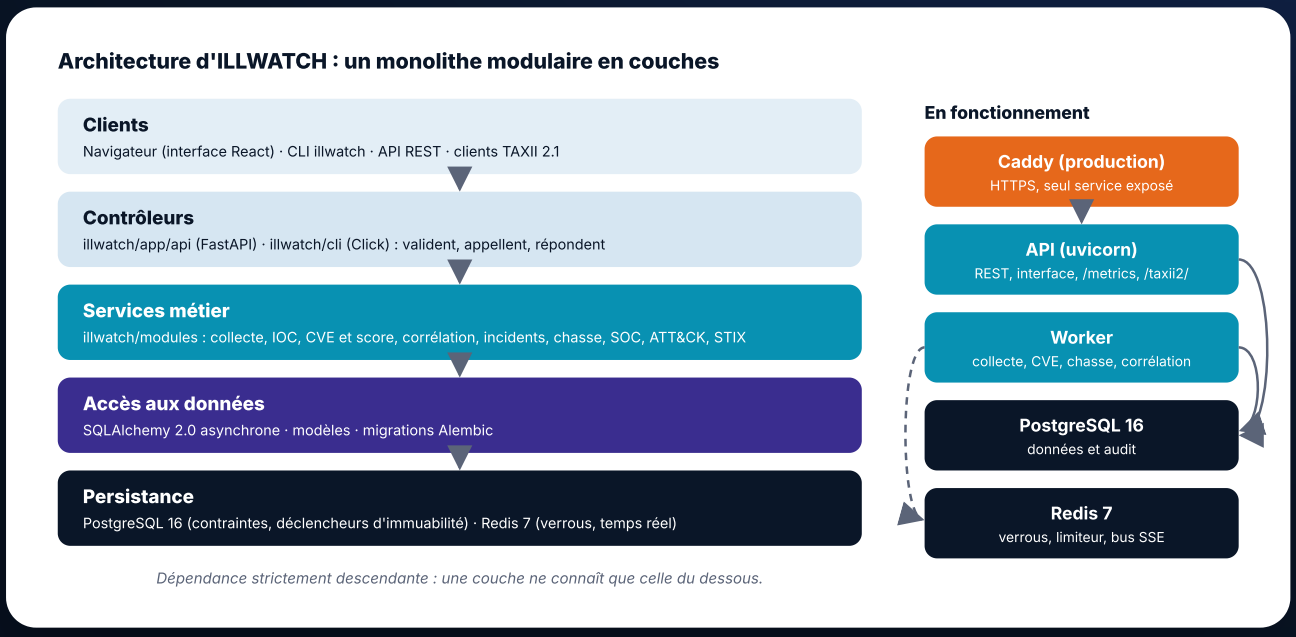
ILL Watch

CAHIER DES CHARGES · VERSION 2.1

Exigences et critères d'acceptation

La spécification opposable d'ILLWATCH : besoins, exigences fonctionnelles RF-01 à RF-61 avec leur statut réel, exigences non fonctionnelles mesurées, architecture, modèle de données, interfaces et risques.



Dépendance strictement descendante : une couche ne connaît que celle du dessous.

VERSION
2.1 · édition du 10 novembre 2026

LOGICIEL
ILLWATCH 0.2.0.dev0

AUTEUR
Godwill FOKA · CyberillSec, une initiative CYBERILL

POUR QUI
Équipe projet, contributeurs, partenaires

Sommaire

1 Historique des versions 1

2 1. Contexte et mission 2

3 2. Besoins et acteurs 3

4 3. Exigences fonctionnelles 4

5 4. Exigences non fonctionnelles 9

6 5. Architecture et choix techniques 10

7 6. Modèle de données 12

8 7. Interfaces 13

9 8. Critères d'acceptation 14

10 9. Risques 15

11 10. Glossaire 16

Version en ligne, toujours à jour : https://godwillfoka.github.io/cyberillsec-illwatch/
Les numéros de page renvoient au pied de page du document.

Projet	ILLWATCH, plateforme de surveillance de la sécurité et de renseignement sur les menaces
Porté par	CyberillSec, une initiative CYBERILL
Auteur et architecte	Godwill FOKA, architecte principal et ingénieur sécurité, Berlin
Version	v2.1 du 10 novembre 2026, qui complète la v2.0 du 30 octobre 2026. Elle remplace la v1.0.0 de juillet 2026.
Dépôt	`gitlab.com/GodwillFoka/cyberillsec-illwatch` (référence), avec un miroir sur GitHub
Statut	approuvé pour le développement : c'est ce document qui fait foi

> **Quand deux documents ne disent pas la même chose**
>
> Une décision d'architecture acceptée (un ADR, dans `docs/adr/`) l'emporte, parce qu'elle tranche la question explicitement. Vient ensuite ce cahier des charges, puis la vision produit. Quand nous découvrons un écart, nous écrivons une nouvelle décision ; nous ne le réglons jamais en silence dans le code. La première version de ce document est conservée en PDF, pour l'histoire du projet.

Historique des versions

Version	Date	Ce qui a changé
v1.0.0	juillet 2026	Première version : six modules, exigences RF-01 à RF-28, 44 tâches réparties sur neuf semaines
v1.x	septembre et octobre 2026	Ajustements décidés au fil de l'eau, chacun dans un ADR (ADR-001 à ADR-028) : calcul de la note, hébergement, changement de nom, interface web…
v2.0	30 octobre 2026	Refonte complète : nouveau nom ILLWATCH (anciennement SENTRY), état réel de chaque exigence, nouvelles exigences des jalons M7 à M11 (RF-29 à RF-52), exigences de performance et de sécurité mesurées, modèle de données et API à jour
v2.1	**10 novembre 2026**	Exigences RF-58 à RF-61 (qualité des alertes, priorisation expliquée, transparence du renseignement, site de présentation), chiffres de qualité mis à jour (plus de 680 tests, couverture d'environ 90 %, 31 décisions)

1. Contexte et mission

1.1 CYBERILL et CyberillSec

CYBERILL est une initiative d'ingénierie et de résilience numérique, fondée à Berlin par Godwill FOKA. Elle compte six branches : CyberillSec pour le renseignement sur les menaces, Cyberill Labs pour la recherche, Cyberill Academy pour la formation, Cyberill Consulting pour le conseil, Cyberill CERT pour la réponse aux incidents et Cyberill Cloud pour les plateformes hébergées.

ILLWATCH est le logiciel au cœur de CyberillSec. Il s'appelait SENTRY jusqu'au 8 octobre 2026, date à laquelle nous avons découvert que ce nom était déjà pris. Seuls les documents historiques gardent l'ancien nom.

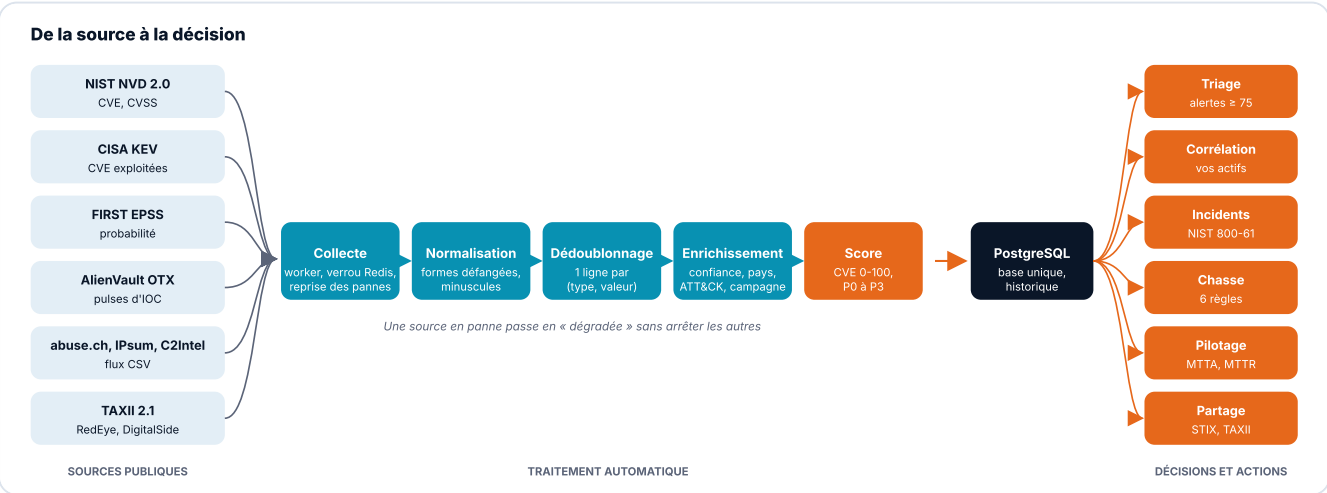
1.2 Le problème

Les équipes de sécurité font face à cinq difficultés qui se renforcent les unes les autres :

1. **Les attaques vont plus vite.** Une faille publiée est souvent exploitée en moins de 48 heures. Corriger une fois par mois ne suffit plus.
2. **Les obligations se durcissent.** En Europe, NIS 2 et DORA imposent une veille active et la déclaration des incidents graves sous 24 à 72 heures.
3. **Les outils coûtent cher.** Une plateforme commerciale de renseignement coûte de 40 000 à 150 000 € par an, hors de portée des PME, des ETI et des collectivités.
4. **Les outils libres sont lourds.** Les plateformes open source complètes demandent plusieurs serveurs et au moins 16 Go de mémoire.
5. **Le bruit épuise les équipes.** Un analyste passe 60 à 70 % de son temps à recopier des indicateurs et à trier des alertes en double.

1.3 La mission

Donner à une équipe d'une à dix personnes ce dont elle a besoin le lundi matin : **savoir quoi corriger d'abord, quoi bloquer, ce qui a changé, et pouvoir prouver ce qui a été fait**. Le tout avec une plateforme open source (licence MIT), que l'on peut héberger en Europe et qui tient dans moins de 256 Mo de mémoire.



2. Besoins et acteurs

2.1 Ce dont les utilisateurs ont besoin

Nous sommes partis de six profils réels et de ce qu'ils attendent concrètement.

Réf.	Qui	Ce dont il a besoin	Ce qu'ILLWATCH lui apporte
BU-01	Analyste SOC (niveaux 1 et 2)	un seul endroit où chaque alerte arrive déjà enrichie et classée	le triage, la note détaillée, la corrélation
BU-02	Analyste en renseignement, chasseur de menaces	filtrer des centaines de milliers d'indicateurs, chercher des traces, exporter	la liste des indicateurs, la chasse, les exports CSV, JSON, STIX et TAXII
BU-03	Responsable de la sécurité	des indicateurs de pilotage et la preuve des actions	l'écran Pilotage (délais de prise en charge et de résolution, objectifs), un audit infalsifiable
BU-04	Administrateur système, DevOps	une installation légère, pilotable par API et en ligne de commande	Docker Compose, une ligne de commande complète, une API REST, des sondes de santé
BU-05	Responsable du parc informatique	décrire l'inventaire sans tout ressaisir	l'import Excel ou CSV et un export que l'on peut réimporter
BU-06	Partenaire (SIEM, CERT)	recevoir le renseignement sans avoir accès à l'interface	un serveur TAXII 2.1 en lecture seule

2.2 Qui intervient

Acteur	Rôle dans l'application	Ce qu'il peut faire
Administrateur	ADMIN	tout, y compris les comptes, les sources, les réglages, les suppressions et l'audit
Analyste	ANALYST	trier, traiter les incidents, chercher des traces, tenir l'inventaire, écarter les faux positifs, suivre les campagnes
Lecteur	VIEWER	consulter ; c'est aussi le rôle d'un partenaire TAXII
Worker	programme automatique	collecter les sources, mettre à jour les failles, corréler, lancer les chasses programmées, faire le ménage
Sources externes		NVD, KEV, EPSS, OTX, abuse.ch, IPsum, C2IntelFeeds, flux TAXII, ATT&CK, DB-IP
Visiteur du site	public	demander une démonstration

Le détail de chaque action, rôle par rôle, se trouve dans le manuel fonctionnel.

3. Exigences fonctionnelles

Chaque exigence porte un numéro (RF-xx), une priorité et un état.

- **Priorité** : P0 indispensable, P1 importante, P2 souhaitable.
- **État** : ✅ **vérifiée** sur données réelles ; 🟢 **livrée**, c'est-à-dire codée, testée et validée par l'intégration continue, mais pas encore vérifiée sur le poste de test ou le serveur ; 🟡 **partielle** ; ⏳ **prévue**.

3.1 Modules de la v1.0 (M1 à M6)

Ce sont les fondations : configuration, collecte des menaces, suivi des failles, incidents, tableau de bord et chasse. Toutes ces exigences sont vérifiées sur données réelles.

Réf.	Module	Ce que la plateforme doit faire	Prio.	État
RF-01	Socle	Lire sa configuration depuis l'environnement ou un fichier `.env`, et la vérifier dès le démarrage	P0	✅
RF-02	Socle	Créer et faire évoluer la base de données par des migrations automatiques	P0	✅
RF-03	Socle	Offrir une ligne de commande pour initialiser, remplir et diagnostiquer	P1	✅
RF-04	Flux	Gérer les sources de menaces : les créer, les consulter, les modifier, les supprimer	P0	✅
RF-05	Flux	Collecter les sources en arrière-plan, sans ralentir l'application	P0	✅
RF-06	Flux	Lire les formats JSON, CSV, STIX 2.1 et TAXII 2.1	P0	✅
RF-07	Flux	Reconnaître et stocker les indicateurs : adresses IPv4 et IPv6, plages réseau, domaines, URL, e-mails, empreintes MD5, SHA-1 et SHA-256	P0	✅
RF-08	Flux	Éviter les doublons : un indicateur déjà connu est mis à jour, pas recréé	P0	✅
RF-09	Flux	Faire tourner un programme qui collecte régulièrement les sources arrivées à échéance	P1	✅
RF-10	Flux	Se connecter à AlienVault OTX	P1	✅
RF-11	Failles	Synchroniser les failles avec la base du NIST (API NVD 2.0), page par page et sans dépasser son débit	P0	✅
RF-12	Failles	Importer le catalogue des failles exploitées de la CISA (KEV)	P0	✅
RF-13	Failles	Importer les probabilités d'exploitation EPSS publiées par FIRST	P0	✅

Réf.	Module	Ce que la plateforme doit faire	Prio.	État
RF-14	Failles	Calculer une note de risque de 0 à 100, toujours identique pour les mêmes données (ADR-001, ADR-014)	P0	✅
RF-15	Failles	Rechercher et filtrer les failles sur plusieurs critères	P1	✅
RF-16	Failles	Lever une alerte quand une faille dépasse le seuil de risque	P1	✅
RF-17	Incidents	Représenter les incidents avec leur gravité	P0	✅
RF-18	Incidents	Faire passer chaque incident par six étapes strictes, selon la méthode NIST SP 800-61	P0	✅
RF-19	Incidents	Tenir pour chaque incident un historique que personne ne peut modifier	P0	✅
RF-20	Incidents	Rattacher des indicateurs et des failles à un incident	P1	✅
RF-21	Tableau de bord	Afficher les chiffres clés en temps réel	P0	✅
RF-22	Tableau de bord	Montrer l'activité récente sur les menaces et les failles	P1	✅
RF-23	Tableau de bord	Donner une vue d'ensemble de l'état du SOC	P1	✅
RF-24	Tableau de bord	Exporter les données en JSON et en CSV	P1	✅
RF-25	Chasse	Appliquer des règles de chasse qui comparent des motifs	P1	✅
RF-26	Chasse	Fournir un catalogue de règles prêtes à l'emploi (cinq demandées, six livrées)	P1	✅
RF-27	Chasse	Lancer une chasse à la demande, selon un calendrier ou depuis un incident	P1	✅
RF-28	Chasse	Garder la trace de chaque chasse et de ce qu'elle a trouvé	P1	✅

3.2 Durcissement et interface (M7)

Ces exigences préparent la mise en production : sécurité des accès, sauvegardes, HTTPS, et l'interface web.

Réf.	Ce que la plateforme doit faire	Prio.	État
RF-29	Tenir un journal d'audit où l'on peut seulement ajouter : connexions, accès refusés, actions d'administration, exports, chasses	P0	🟢

Réf.	Ce que la plateforme doit faire	Prio.	État
RF-30	Séparer les rôles dans PostgreSQL : seul le propriétaire fait les migrations, l'application n'a aucun droit sur la structure	P0	🟢
RF-31	Permettre de révoquer une session : jeton d'accès de 15 minutes, jeton de renouvellement à usage unique, réutilisation détectée	P0	🟢
RF-32	Limiter les tentatives de connexion, par compte et adresse, par compte et par adresse	P0	🟢
RF-33	Exposer une sonde de vie (`/health`) et une sonde de disponibilité (`/ready`)	P1	🟢
RF-34	Faire des sauvegardes vérifiées et tester leur restauration	P0	🟢
RF-35	Se déployer en HTTPS derrière un reverse proxy, avec les migrations dans un conteneur temporaire	P0	🟡 vérifié en local ; le serveur européen reste à louer
RF-36	Offrir une interface web en temps réel (ADR-016) : vue d'ensemble, triage, incidents, indicateurs, failles, chasse, sources, corrélations, inventaire, campagnes, pilotage, administration	P0	🟢
RF-37	Parler cinq langues (français, anglais, allemand, espagnol, chinois), choisies selon le pays de l'utilisateur (ADR-027)	P1	🟢

3.3 Détection et corrélation (M8)

Ici, ILLWATCH cesse de parler des menaces « en général » pour parler de **votre** parc.

Réf.	Ce que la plateforme doit faire	Prio.	État
RF-38	Donner à chaque indicateur une note de confiance, selon la fiabilité des sources, sa fraîcheur et sa récurrence (ADR-017)	P0	✅
RF-39	Situer les adresses IP (pays, opérateur) sans interroger de service extérieur (ADR-019)	P1	✅
RF-40	Tenir un inventaire des équipements : produit, éditeur, version, plateforme, criticité, adresses, domaines (ADR-020)	P0	✅
RF-41	**Importer et exporter l'inventaire en Excel ou en CSV, avec un aperçu, et n'importer que si tout le fichier est correct (ADR-028)**	P0	🟢
RF-42	Comparer l'inventaire aux failles et aux indicateurs (règles CORR-01 à CORR-03), justifier chaque résultat et tenir compte des versions (ADR-020, ADR-021)	P0	✅
RF-43	Ne jamais classer sous P1 une faille déjà exploitée ; si la donnée EPSS manque, garder la dernière connue (ADR-014)	P0	✅

3.4 Opérations SOC (M9)

Ces exigences organisent le travail de l'équipe de sécurité au quotidien.

Réf.	Ce que la plateforme doit faire	Prio.	État
RF-44	Répartir les incidents entre trois niveaux d'analystes, avec une escalade motivée et inscrite dans l'historique (ADR-022)	P0	🟢
RF-45	Gérer les faux positifs : une liste d'indicateurs exclus, et un évènement écarté ne revient jamais	P0	🟢
RF-46	Lancer une chasse depuis un incident et proposer des actions adaptées à chaque étape	P1	🟢
RF-47	Mesurer le travail : délais de prise en charge et de résolution, objectifs par gravité, retards, charge de chaque analyste	P1	🟢

3.5 Renseignement contextuel (M10)

Ces exigences replacent chaque alerte dans son contexte : qui attaque, comment, avec quelle infrastructure.

Réf.	Ce que la plateforme doit faire	Prio.	État
RF-48	S'appuyer sur le référentiel MITRE ATT&CK : technique liée à chaque détection, acteur cité (ADR-023)	P1	🟢
RF-49	Suivre des campagnes : infrastructure rattachée, acteur, attribution à partir de l'infrastructure (ADR-024)	P1	🟢
RF-50	Partager le renseignement : export STIX 2.1 et serveur TAXII 2.1 en lecture seule (ADR-025)	P1	🟢

3.6 Observabilité et déploiement (M11)

Réf.	Ce que la plateforme doit faire	Prio.	État
RF-51	Publier ses métriques pour Prometheus (`/metrics`), avec des tableaux Grafana et dix alertes (ADR-026)	P1	🟢
RF-52	Disposer de trois environnements (développement, préproduction, production), d'un test d'acceptation automatique et de sept jours de préproduction sans intervention	P0	🟡 le serveur reste à louer

3.7 Qualité, transparence et présentation (après M11)

Ces exigences viennent de la revue d'amélioration du 30/10/2026 et du site de présentation du 22/10/2026. Elles ne changent pas les critères de sortie des jalons.

Réf.	Ce que la plateforme doit faire	Prio.	État
RF-58	Mesurer la qualité des alertes en situation réelle : pour chaque règle et chaque source, part des évènements confirmés, écartés comme faux positifs ou encore ouverts (ADR-029)	P1	🟢
RF-59	Expliquer la priorité de chaque évènement corrélé : criticité de l'actif, exposition, exploitation connue, version, échéance (ADR-030)	P1	🟢

Réf.	Ce que la plateforme doit faire	Prio.	État
RF-60	Montrer pour chaque donnée sa source, sa fraîcheur et sa confiance, et signaler une donnée périmée (ADR-031)	P1	🟢
RF-61	Présenter la solution sur un site public en cinq langues, avec un formulaire de demande de démonstration (ADR-027)	P2	🟢

3.8 Exigences prévues après la v0.2.0

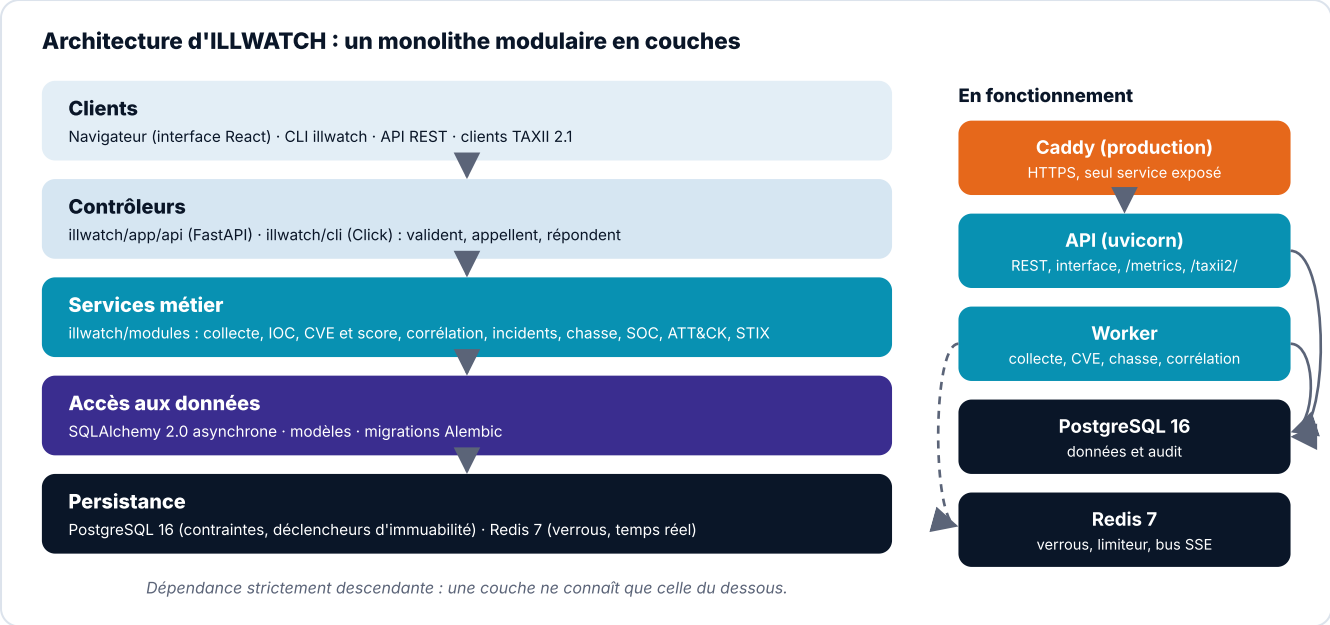
Réf.	Ce que nous prévoyons	Prio.
RF-53	Un assistant d'analyse, fondé sur un modèle de langage, pour lire les bulletins et les incidents	P2
RF-54	Plusieurs organisations dans une même instance, et la connexion unique (OIDC)	P1
RF-55	Des connecteurs vers les outils de gestion de parc (GLPI, CMDB) pour remplir l'inventaire	P2
RF-56	L'import de règles Sigma depuis des collections TAXII	P2
RF-57	La prise en compte du type de faille (CWE) dans la note, avec de nouveaux poids (décision à écrire)	P2

4. Exigences non fonctionnelles

Ce sont les qualités attendues de la plateforme : sécurité, rapidité, sobriété, fiabilité. Pour chacune, nous indiquons ce que nous avons réellement mesuré.

Réf.	Domaine	Ce qui est exigé	Ce que nous avons mesuré
RNF-SEC-01	Sécurité	HTTPS en production, aucun secret dans le code ni dans la base	Caddy en TLS 1.3 vérifié en local ; gitleaks ne trouve aucun vrai secret ; clés fournies sous forme de modèles
RNF-SEC-02	Sécurité	Toutes les entrées vérifiées, aucune requête SQL construite à la main	Pydantic refuse tout champ inattendu ; l'analyse Semgrep ne relève rien ; bandit est vert
RNF-SEC-03	Sécurité	Les collectes ne peuvent pas être détournées vers le réseau interne (SSRF)	seules les adresses publiques sont autorisées, le DNS est vérifié, chaque redirection est revérifiée
RNF-SEC-04	Sécurité	Aucune faille critique dans l'image Docker	zéro faille critique (analyse du 7 octobre 2026)
RNF-SEC-05	Sécurité	Les fichiers importés sont bornés en taille, en lignes et en contenu d'archive	inventaire : 2 Mo, 5 000 lignes, 40 Mo une fois décompressé
RNF-PERF-01	Rapidité	95 % des listes de failles affichées en moins de 250 ms	8 ms avec filtres, 108 ms avec recherche, sur 30 000 failles
RNF-PERF-02	Rapidité	1 000 indicateurs ingérés en moins de 5 secondes	vérifié par un test à chaque modification
RNF-PERF-03	Rapidité	Tableau de bord en moins de 1 s, export de 500 000 indicateurs en moins de 15 s	0,63 s et 12,0 s avec 528 453 indicateurs (8 octobre 2026)
RNF-MEM-01	Sobriété	L'application tient dans moins de 256 Mo	120 Mo au plus fort, en ingérant 37 000 indicateurs
RNF-DISP-01	Disponibilité	99,5 % du temps, hors maintenance	à mesurer pendant les sept jours de préproduction
RNF-MAIN-01	Maintenabilité	Aucune erreur de style ni de typage, au moins 80 % du code couvert par les tests	environ 90 % ; plus de 680 tests (CI du 30/10/2026)
RNF-MAIN-02	Maintenabilité	Chaque choix important expliqué dans une décision écrite (ADR)	31 décisions
RNF-ACC-01	Accessibilité	Site de présentation conforme au niveau AA des WCAG 2.2	vérifié par axe-core à chaque modification
RNF-EDIT-01	Écriture	Ni tiret long ni tiret moyen dans le dépôt	vérifié par le test `test_ponctuation.py`

5. Architecture et choix techniques

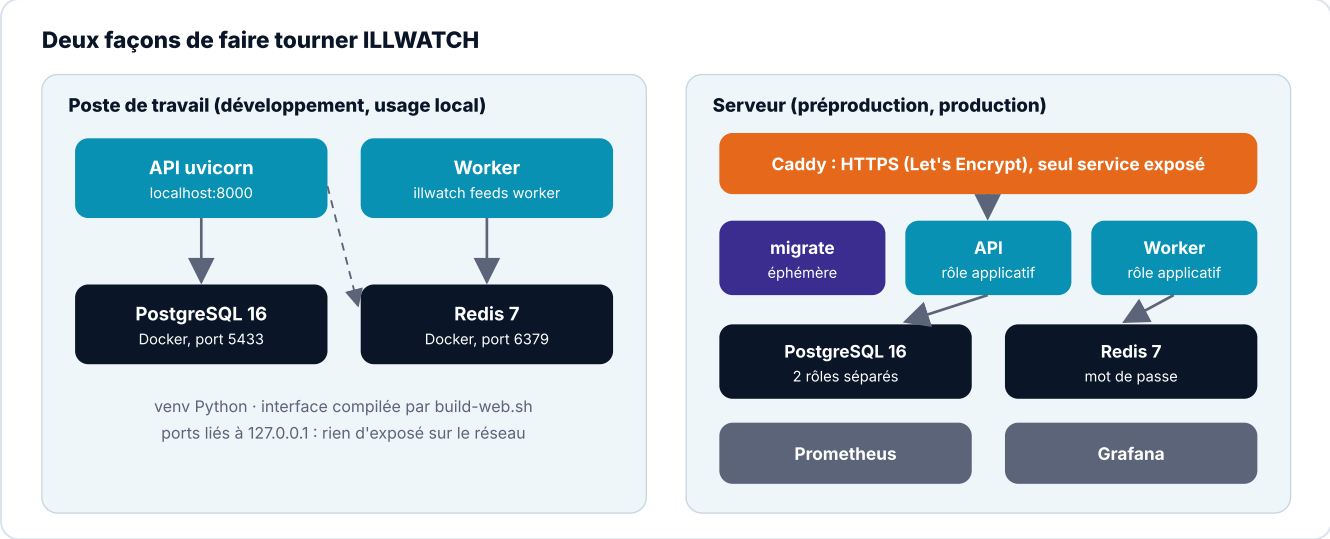


ILLWATCH est un **monolithe modulaire organisé en couches**. Concrètement, cela veut dire trois choses. Chaque couche ne dépend que de celles qui sont en dessous d'elle. Les points d'entrée (l'API et la ligne de commande) ne contiennent aucune règle métier : ils transmettent. Et les deux mécanismes les plus sensibles, le calcul de la note et l'enchaînement des étapes d'un incident, sont des fonctions pures, testées à 100 %. Le détail est dans Architecture.

Couche	Ce que nous utilisons	Pourquoi
Langage	Python 3.12 et 3.14	c'est la langue de l'écosystème du renseignement (STIX, TAXII, YARA, Sigma)
API	FastAPI et Pydantic v2	des entrées vérifiées strictement, une documentation générée automatiquement
Données	SQLAlchemy 2.0 asynchrone, asyncpg, PostgreSQL 16, Alembic	la base elle-même fait respecter les règles (contraintes, déclencheurs)
Cache et temps réel	Redis 7	verrous du worker, limitation des connexions, diffusion des évènements
Interface	React 19, TypeScript, Vite	des types générés à partir de la description de l'API
Fichiers bureautiques	openpyxl	lire et écrire des fichiers Excel en Python pur (ADR-028)
Déploiement	Docker Compose et Caddy	HTTPS automatique, un seul service exposé
Supervision	Prometheus et Grafana	des métriques sans dépendance supplémentaire dans l'application

Ce que nous avons volontairement écarté. Les microservices, parce qu'un monolithe bien découpé supporte 100 000 évènements par jour. Elasticsearch, parce qu'un PostgreSQL bien indexé suffit. Un

moteur Sigma complet, parce que six règles simples et prévisibles suffisent pour analyser des observables.



6. Modèle de données

La base compte 28 tables, créées par 20 migrations. À chaque modification, l'intégration continue vérifie qu'elles s'appliquent, s'annulent et se réappliquent sans erreur. Les règles métier importantes sont garanties par la base elle-même : clés étrangères, unicité, valeurs autorisées pour chaque liste, et déclencheurs qui empêchent de modifier les historiques.

Domaine	Tables
Comptes et sécurité	`users`, `refresh_tokens`, `audit_events` (ajout seulement)
Renseignement	`threat_feeds`, `collection_runs`, `collector_state`, `indicators`, `indicator_sources`, `exclusions`
Enrichissement	`enrichment_datasets`, `ip_ranges`
Failles	`cves`, `cve_priority_history`, `cve_alerts`
Incidents	`incidents`, `incident_events` (ajout seulement), `incident_indicators`, `incident_cves`
Chasse	`hunting_sessions`, `hunting_matches`
Corrélation	`assets`, `correlated_events`
Contexte des attaquants	`attack_techniques`, `attack_actors`, `attack_uses`, `campaigns`, `campaign_indicators`
Site	`demo_requests`

Les index qui comptent le plus : l'unicité de chaque indicateur (type et valeur), sa date de dernière observation pour gérer l'expiration, la note et la priorité des failles, et l'ordre chronologique des incidents.

7. Interfaces

ILLWATCH s'utilise de plusieurs façons, selon que l'on est un humain, un script ou un autre outil.

Interface	Ce qu'elle offre
Application web	douze écrans, mise à jour en temps réel, cinq langues
API REST `/api/v1`	77 routes ; une documentation interactive sur `/docs`, hors production
Ligne de commande `illwatch`	71 commandes, regroupées par thème : `db`, `users`, `feeds`, `cves`, `assets`, `correlate`, `incidents`, `hunt`, `soc`, `campaigns`, `attack`, `enrich`, `stix`, `taxii`, `audit`, `dashboard`, `status`…
TAXII 2.1 `/taxii2/`	deux collections en lecture seule : les indicateurs et les campagnes
Métriques `/metrics`	au format Prometheus, protégées par un jeton ou réservées au réseau interne
Webhook	une notification pour chaque alerte sur une faille
Fichiers	inventaire Excel ou CSV ; exports CSV, JSON et STIX 2.1 ; sauvegardes PostgreSQL

8. Critères d'acceptation

Voici comment nous vérifions que la plateforme fait ce qu'elle promet.

Ce que nous vérifions	Comment	Résultat
Le travail d'un SOC, de bout en bout	48 contrôles (`scripts/scenario_soc.py`), sur données réelles et en intégration continue	✅ 48 sur 48
Les jalons M1 à M6	la commande `illwatch status`, sur données réelles	✅
La corrélation, sur un jeu de test figé	tous les cas attendus détectés, taux de faux positifs publié	✅ 100 % détectés, 0 % de faux positifs
Un incident traité du niveau 1 au niveau 3 sans quitter ILLWATCH, délai mesuré	test d'acceptation	🟢 vérifié en intégration continue
La technique ATT&CK et l'acteur affichés sur une alerte	test d'acceptation	🟢 vérifié en intégration continue
Un déploiement HTTPS sur un serveur européen, noté A par SSL Labs	workflow « Clôture M7 »	⏳
Sept jours de préproduction sans intervention, restauration testée	workflow et journal du serveur	⏳
Un inventaire de 1 000 lignes importé depuis Excel, sans erreur ni perte	tests `test_inventory_file.py` et capture d'écran en intégration continue	🟢

9. Risques

Aucun projet n'est sans risque. Voici ceux que nous surveillons et ce que nous faisons pour les réduire.

Réf.	Risque	Gravité	Ce que nous faisons
RSK-01	Les API externes limitent leur débit (la NVD en particulier)	élevée	clés gratuites, attente progressive entre les essais, reprise là où l'on s'était arrêté
RSK-02	Une source externe tombe en panne	moyenne	chaque collecteur est isolé ; une source en échec est réessayée de plus en plus rarement
RSK-03	Le volume d'indicateurs explose	élevée	expiration selon le type, index, pagination efficace
RSK-04	Le périmètre dérive	moyenne	des jalons dont la fin se constate ; les nouvelles idées attendent dans une liste
RSK-05	Une modification casse quelque chose	moyenne	deux chaînes d'intégration continue et une vérification sur données réelles chaque semaine
RSK-06	Le service EPSS est indisponible	moyenne	plancher pour les failles exploitées et dernière valeur connue (ADR-014)
RSK-07	L'inventaire est mal renseigné	moyenne	aide à la saisie, comparaison des versions, mesure sur un jeu de test
RSK-08	Un fichier importé est malveillant	moyenne	limites de taille, contrôle des archives, refus expliqués ligne par ligne
RSK-09	Le projet repose sur une seule personne	élevée	documentation, décisions écrites et tests ; ouverture aux contributeurs

10. Glossaire

Tous les termes techniques sont expliqués dans le glossaire du manuel fonctionnel.